



FINAL YEAR PROJECT REPORT

LeadBot

An AI lead-qualification chatbot for small businesses

A website chatbot that answers from a business's own knowledge base in English and Roman Urdu, captures and scores leads, and gives the owner a pipeline, an audit log and four ways to train the bot.

Student	Muhammad Ali
Roll number	<i>add in build_report.py</i>
Supervisor	<i>add in build_report.py</i>
Department	<i>add in build_report.py</i>
University	<i>add in build_report.py</i>
Date	October 2026
Live demo	https://leadbot-g3d4.onrender.com
Source code	https://github.com/MAliQ-pak/leadbot

Contents

Abstract

1 Introduction

2 Background

3 Requirements

4 System design

5 Implementation

6 Testing and evaluation

7 Deployment

8 Limitations and future work

9 Conclusion

References

Appendix A: Viva questions and answers

Abstract

LeadBot is a working website chatbot that answers customer questions from a business's own knowledge base, collects lead details in conversation, and scores each lead as hot, warm or cold with a stated reason. It is deployed at a public link and demonstrated on a fictional solar installer, BrightPath Solar.

Replies are produced by retrieval-augmented generation: a TF-IDF retriever over character n-grams finds the relevant knowledge-base topics, and a large language model (Google Gemini, or Anthropic Claude) answers only from them. A rule-based offline bot takes over whenever no AI is available, so the demo never depends on internet or payment. Lead scores blend explainable rules with a logistic-regression classifier.

A business owner manages everything from a password-protected admin panel: a deals pipeline board, leads by temperature, full conversations, notes, an audit log, and four ways to train the bot (Q&A pairs, unanswered questions, uploaded documents, imported web pages). In testing, the classifier reached 92% accuracy in 5-fold cross-validation on a 62-example illustrative dataset, and the AI answered in English and Roman Urdu without inventing prices or accepting a fake discount.

1 Introduction

1.1 Problem

Small businesses lose sales because website visitors ask questions outside office hours, get no answer, and leave without sharing contact details. When enquiries do arrive, the owner must read each one to decide who is a serious buyer. In Pakistan this is harder: customers often write in Roman Urdu, which simple keyword bots handle poorly.

1.2 Aim

Build a chatbot that a small business can put on its website to answer questions from its own information, capture and qualify leads automatically, and hand the owner a ranked list of who to call first.

1.3 Objectives

1. Answer visitor questions only from the business's knowledge base, never inventing prices or policies.
2. Support English and Roman Urdu.
3. Extract lead details (name, phone, email, need, budget, timeline) from natural conversation.
4. Score each lead 0-100 with a hot, warm or cold tier and a human-readable reason, combining rules with a trained classifier.
5. Give the owner an admin panel to manage leads and to train the bot without editing code.
6. Keep working with no internet or AI access, so a demonstration cannot fail.
7. Deploy the system at a public link with basic security.

1.4 Scope

The system targets one business at a time, configured through a Markdown knowledge base; the demo business, BrightPath Solar, is fictional. It covers a website widget, not WhatsApp or phone channels. CRM integration was built as an optional webhook but is switched off for this project, with leads managed in the admin panel instead.

1.5 Report structure

Section 2 reviews the background. Sections 3 to 5 cover requirements, design and implementation. Section 6 evaluates the system, Section 7 describes deployment, and Sections 8 and 9 discuss limitations and conclude. Appendix A prepares answers for the viva.

2 Background

LeadBot combines three established ideas: retrieval-augmented generation for grounded answers, classic TF-IDF retrieval for finding the right text, and lead scoring for sales prioritisation. References marked [R#] are listed in the References section and must be checked before submission.

2.1 Website chatbots

Early website bots followed fixed decision trees or keyword rules: predictable, but brittle when visitors phrase things differently. Large language models (LLMs) understand free text and reply fluently, but on their own they can state plausible facts that are wrong, such as an invented price. For a sales bot, that risk is unacceptable.

2.2 Retrieval-augmented generation (RAG)

RAG retrieves relevant passages first and asks the model to answer from them [R1]. The model's knowledge of the business then comes only from text the owner controls, and updating the bot means updating that text, not retraining a model. LeadBot adds an explicit rule: if the retrieved text does not cover the question, the bot says it is not sure and offers a human callback.

2.3 TF-IDF retrieval

TF-IDF weights a term by how often it appears in a passage, discounted by how common it is across all passages [R2]. Cosine similarity between the question's vector and each passage's vector ranks the passages. LeadBot uses character n-grams (3 to 5 characters) instead of whole words, so spelling variants common in Roman Urdu, such as "qeemat" and "keemat" (price), still share most of their n-grams. The vectoriser comes from scikit-learn [R3].

Embedding-based retrieval with a vector database is the modern alternative. It handles synonyms better but needs an embedding model or API. For a knowledge base of tens of topics, TF-IDF is fast, free, offline and explainable, which suits this project.

2.4 LLM APIs

Hosted LLMs are called over HTTPS with a secret API key. LeadBot supports two providers: Google Gemini, which offers a free tier [R4], and Anthropic Claude, which is paid per token [R5]. Both are asked to return JSON containing the reply and any lead details, which the server then validates.

2.5 Lead qualification and scoring

Sales teams rank leads so effort goes to the most likely buyers. Rule-based scoring adds points for signals such as a shared phone number or a stated budget; it is transparent but its weights are guesses. Predictive scoring learns weights from past outcomes, often with logistic regression [R6], whose coefficients can still be read and explained. LeadBot uses both and shows each score separately.

3 Requirements

The system serves two users: the **website visitor**, who chats anonymously, and the **business owner**, who logs into the admin panel.

3.1 Functional requirements

ID	Requirement	User
FR1	Chat widget embeddable on any website with one script tag	Visitor
FR2	Answer questions only from the knowledge base; say "not sure" otherwise	Visitor
FR3	Reply in the visitor's language (English or Roman Urdu)	Visitor
FR4	Extract name, phone, email, need, budget and timeline from chat	System
FR5	Score each lead 0-100 with a tier (hot, warm, cold) and a reason	System
FR6	List, filter and search leads; view full conversations	Owner
FR7	Move deals through six stages on a pipeline board; add notes	Owner
FR8	Train the bot: Q&A pairs, unanswered questions, documents, web pages, main text	Owner
FR9	Test what the bot would retrieve for any question	Owner
FR10	Optional push of warm and hot leads to a CRM webhook	System
FR11	Audit log of owner actions and key system events, filterable by type	Owner

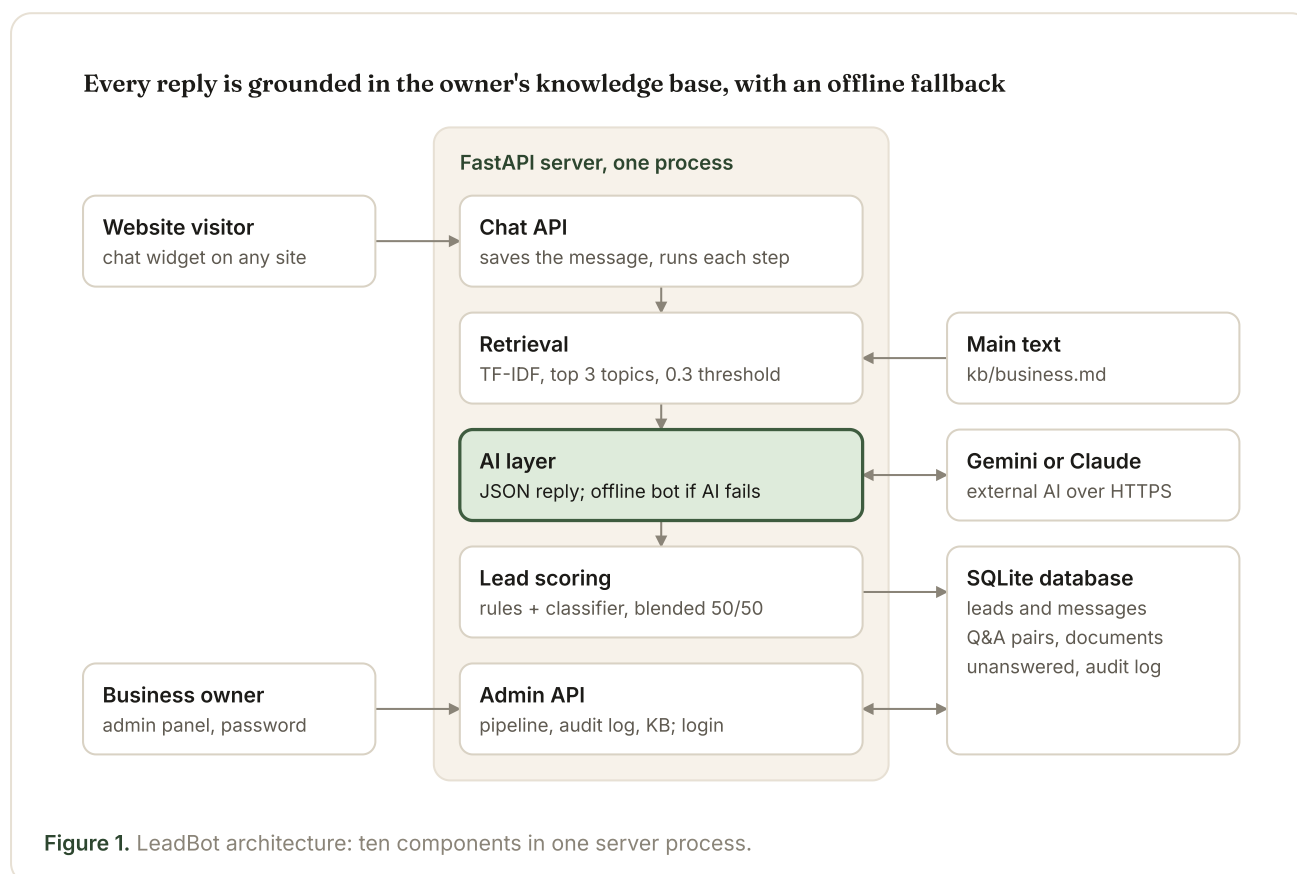
3.2 Non-functional requirements

ID	Requirement	How it is met
NFR1	Works offline	Rule-based bot answers when no AI key, quota or internet is available
NFR2	Explainable	Every score shows its rule reasons and the model's probability
NFR3	Secure admin	Owner login; admin locked to localhost if no password is set
NFR4	Controlled cost	Limits of 200 AI replies per hour and 30 per conversation
NFR5	Responsive	Pages work at phone width (375 px) with no sideways scrolling
NFR6	Fast replies	Typical AI reply in 1-4 s; 15 s timeout before falling back
NFR7	Simple to run	One command locally; one Blueprint file on Render

4 System design

LeadBot is one Python web server (FastAPI) that serves the website, the chat API and the admin panel, with a single SQLite database. Keeping everything in one process makes it simple to run, deploy and explain.

4.1 Architecture



A chat message runs down the server column; retrieval also reads the Q&A pairs and documents stored in the database. If the external AI fails, the AI layer answers with the offline bot.

4.2 What happens to one chat message

1. The widget sends the visitor's message and a random session ID to `POST /api/chat`.
2. The server stores the message and loads the last 12 messages of that session.
3. The retriever scores every knowledge-base topic and keeps the top 3. A short follow-up such as "and the inverter?" is retried together with the previous question.
4. If the best match is below 0.3 and the message is a question, the server treats it as unknown: it logs it as **unanswered** for the owner and sends the AI no context, so the AI says it is not sure.
5. The AI provider (Gemini or Claude) receives the rules, the details already known about the visitor, the retrieved topics and the conversation. It returns JSON with the reply and any lead details. On any API problem, the offline bot answers instead.

6. The rules and the classifier each score the lead; the blended score sets the tier.
7. The reply, the lead and the score are saved, and the reply is returned to the widget.

4.3 Data model

Table	Holds	Key columns
<code>messages</code>	Every chat message	<code>session_id</code> , role (user / assistant), content, created
<code>leads</code>	One row per visitor session	name, phone, email, need, budget, timeline, score, rule_score, ml_score, tier, reason, status (deal stage), notes
<code>kb_faq</code>	Q&A pairs added by the owner	question, variants, answer, source (manual / unanswered)
<code>kb_docs</code>	Uploaded documents and imported web pages	name, kind (file / url), origin, text
<code>unanswered</code>	Questions the bot could not answer confidently	<code>session_id</code> , question, score, status (open / answered / dismissed)
<code>audit</code>	Owner actions and key system events	created, actor, kind (lead / kb / auth / system), action, target, detail

The main business text lives in `kb/business.md`, where each `##` heading is one topic. New columns are added with `ALTER TABLE` at start-up, so an existing database keeps working after an upgrade.

4.4 Technology choices

Choice	Why
Python + FastAPI	Short, readable code; request validation built in
SQLite	No database server to install; one file
scikit-learn	TF-IDF retrieval and logistic regression in a few lines
Gemini / Claude APIs	Strong multilingual replies; Gemini has a free tier
Plain HTML, CSS, JavaScript	No build step; every file can be explained line by line
Render	Free hosting directly from the GitHub repository

5 Implementation

The code is about 1,300 lines of Python and 1,600 of front-end code, split into small single-purpose files. The table maps each file to its job.

File	Job
app/main.py	API routes, the chat pipeline, score blending
app/rag.py	Knowledge-base loading and TF-IDF retrieval
app/llm.py	Prompt, Gemini and Claude calls, offline bot, limits
app/scoring.py	Rule-based lead score
app/classifier.py	Classifier features and prediction
app/ingest.py	Reading PDF, Word, TXT, Markdown and web pages
app/auth.py	Owner login
app/db.py	SQLite storage
train_classifier.py	Trains and evaluates the classifier
static/	Website, chat widget, bot tester, admin panel

5.1 Website and chat widget

The demo website for BrightPath Solar shows services, packages, the installation process, warranty and financing. Every price and policy on it matches `kb/business.md`, so the site and the bot never disagree. The chat widget is one JavaScript file that any website loads with a single `<script>` tag. It keeps the visitor's session ID in the browser, restores earlier messages after a page reload, and offers quick-reply buttons. Buttons on the site, such as "Ask about this" on a package, open the chat and send their question.

5.2 Retrieval

Each `##` section of the main text, each Q&A pair, and each topic cut from an uploaded document becomes one chunk. Chunks are vectorised with TF-IDF over character n-grams of length 3 to 5. A synonym table expands Roman Urdu and casual words before matching, for example "kist" to "installment financing" and "timings" to "working hours contact". Chunks whose title words appear in the question get a bonus of up to 0.3, ignoring filler words such as "do" and "you". A best score below 0.3 counts as "not confident".

5.3 AI providers and the offline bot

The environment setting `LLM_PROVIDER` selects Gemini (`gemini-3.8-flash`) or Claude (`claude-sonnet-5-5`). The prompt tells the model to answer only from the retrieved context, reply in the visitor's language and script, ask for one missing detail at a time in a fixed order, ask for email at most

once, and never record a quoted price as the visitor's budget. Gemini is asked for JSON output directly, so its replies always parse.

The system is built to fail safely:

- If the main Gemini model is overloaded or out of free quota, lighter models are tried in turn (`gemini-3.5-flash-lite` , then `gemini-flash-lite-latest`).
- After any API error, the AI is skipped for 60 seconds, so visitors get instant offline replies instead of repeated waits.
- Gemini requests use IPv4 only, because on the development network the IPv6 route hung for about 10 seconds before failing.
- The offline bot extracts details with regular expressions, answers from the best-matching chunk, and asks the next missing question.

5.4 Lead scoring

The rule score adds points for each signal and caps the total at 100:

Signal	Points
Phone shared	25
Email shared	15
Need stated	15
Budget stated	15
Timeline stated	10
Asked about price or buying	10
Name shared	5
Urgent wording ("this week", "jaldi")	5

The classifier is a logistic regression on 11 features, each scaled 0 to 1: the six "detail shared" flags, price intent, urgency, "not now" wording ("just checking", "next year", "baad mein"), number of questions and message length. It outputs the probability that the lead is qualified. The final score blends the two, with the weight `w` set by `ML_WEIGHT` (default 0.5):

$$\text{score} = (1 - w) \times \text{rules} + w \times 100 \times P(\text{qualified})$$

A score of 70 or more is hot, 40 to 69 is warm, and below 40 is cold. If no trained model file exists, the rules alone decide.

5.5 Admin panel

The admin panel has five pages, refreshed every 5 seconds:

- **Overview:** totals by tier and stage, the hot leads to call now, the pipeline and the latest conversations.
- **Pipeline:** a deals board in the style of a CRM, with one column per stage: New, Contacted, Site visit booked, Quote sent, Won and Lost. The middle two follow BrightPath's real sales process, a free site visit followed by a written quote. Each card shows the lead's name, need, budget, phone and a hot, warm or cold tag with its score, hottest first. Dragging a card to another column moves the deal; clicking it opens the lead. Visitors who never shared a phone or email are hidden by default, because they cannot yet be followed up.
- **Leads:** tabs for all, hot, warm, cold, won and lost, plus search. Opening a lead shows its details, the rules and model scores, the reason, the conversation, six stage buttons, notes and a delete button.
- **Audit log:** a dated record of every owner action (stage changes, notes, deletes, knowledge-base edits, uploads, logins and failed logins) and key system events (a new lead, a lead turning hot, an AI failure), filterable by type. Each entry records who acted. A login is logged at most once per 30 minutes and failed logins at most once a minute per address, so the log stays readable.
- **Bot & knowledge:** the active AI and model, a test box, and the training tabs described next.

A lead's tier (hot, warm, cold) and its stage are deliberately separate: the tier is what the visitor showed in the chat, and the stage is what the owner has done since.

5.6 Training the knowledge base

The owner can teach the bot in four ways, and every change rebuilds the index so it applies to the very next message:

1. **Q&A pairs:** a question, other ways to ask it (for example in Roman Urdu) and the answer.
2. **Unanswered questions:** questions the bot could not answer are listed with how often they were asked. The owner types an answer once and it becomes a Q&A pair.
3. **Documents and web pages:** PDF, Word, TXT or Markdown files up to 5 MB, or a page imported by URL, split into topics at headings and roughly every 700 characters.
4. **Main text:** direct editing of `kb/business.md`.

The test box shows which topics a question matches, with a match percentage and whether the bot will answer or say it is not sure.

5.7 Security and cost control

- The admin panel, the bot tester and all admin APIs need the owner's login (HTTP Basic authentication, compared in constant time). Every login and failed login is written to the audit log.
- If no password is set, admin pages open only on the computer running the server, so a deployment cannot be left open by accident.
- API keys and the password live in environment variables, never in the code or the repository.
- Chat replies are written into the page as text, never as HTML, so a reply cannot inject a script.
- Spending limits: 200 AI replies per hour site-wide and 30 per conversation; beyond either, the offline bot answers.

5.8 User interface design

All four interfaces (website, chat widget, bot tester and admin panel) follow one design specification: a serif display font over a clean sans-serif body, and a warm palette built around forest green on cream. One set of colour tokens is shared across the stylesheets, so every page reads as one product.

Typography

Font	Weights	Used for
Fraunces	600	Headings, the logo, lead names in the admin panel, the chat widget title
Fraunces	600 italic	Emphasised words, such as "rooftop solar" in the hero and "Solar" in the logo
Inter	400	Body text, chat messages, table cells
Inter	500	Navigation, labels, tabs
Inter	600	Buttons, column and card titles
Inter	700	Numbers: scores, prices, totals

Both fonts are served from the project's own `static/fonts` folder (open-licence variable fonts, one file covering every weight), so every page, and this report, shows them with no internet connection.

Colour scheme

Role	Hex	Used for
Forest green	 #3E5C3F	Primary buttons, links, accents, the visitor's chat bubbles, "warm" tag
Dark green	 #2D462F /  #243A27	Button hover, chat widget header, stats strip, contact banner
Ink	 #1C1B18	Main text, admin sidebar
Body text	 #4B463D /  #5F5A51	Paragraphs and secondary text
Muted text	 #8A8478 /  #A39B8B	Hints, timestamps, empty states
Cream	 #F7F2EA /  #FFFDF8	Page backgrounds and cards
Borders	 #E7E0D4 /  #EDE6D8	Card, table and input outlines
Green tint	 #DEEBDB /  #E8F0E4	Quick-reply chips, row hover, hero gradient (#EEF3E9 to #E5EEDA)
Blue	 #2B5CE6	The classifier's bar in score breakdowns
Sand and brown	 #F2E3CF /  #7A5A2A	"Hot" tag, Quote sent stage, Q&A source labels
Navy	 #2E3A6E on  #EEF3FB	New stage, document labels, solar panels in the illustration
Purple	 #4C36D6 on  #ECE9FB,  #5E3A6E on  #E8DCEB	Contacted and Site visit booked stages

Role	Hex	Used for
Destructive red	 hsl(0 84% 60%)	Delete buttons and the unanswered-questions alert only

The specification has no red or amber for lead temperature, so hot, warm and cold use its own tones: brown for hot, green for warm and grey (#6F695F) for cold. Red is kept for destructive actions, so it always means "careful".

6 Testing and evaluation

Every feature was tested against the running system: the classifier by cross-validation, retrieval by scoring covered and uncovered questions, the AI by scripted conversations, and security by direct requests. The results support the design, with one caveat: the classifier's data and the conversation set are small and written for this project.

6.1 Lead classifier

The training set has 62 hand-labelled example leads: 27 qualified and 35 not. They were written to cover realistic cases the rules miss, such as a visitor who shares a phone number but says "just checking prices for next year". The model was evaluated with stratified 5-fold cross-validation, so each lead was predicted by a model that never saw it.

Class	Precision	Recall	F1	Leads
Not qualified	0.89	0.97	0.93	35
Qualified	0.96	0.85	0.90	27

Overall accuracy was 0.92. Because the labels reflect the author's own judgement, this shows the model learned that judgement consistently, not that it predicts real sales.

Not-now wording pulls a lead down hardest; a shared phone number lifts it most

Logistic-regression weight per feature. Right of zero = more likely qualified.

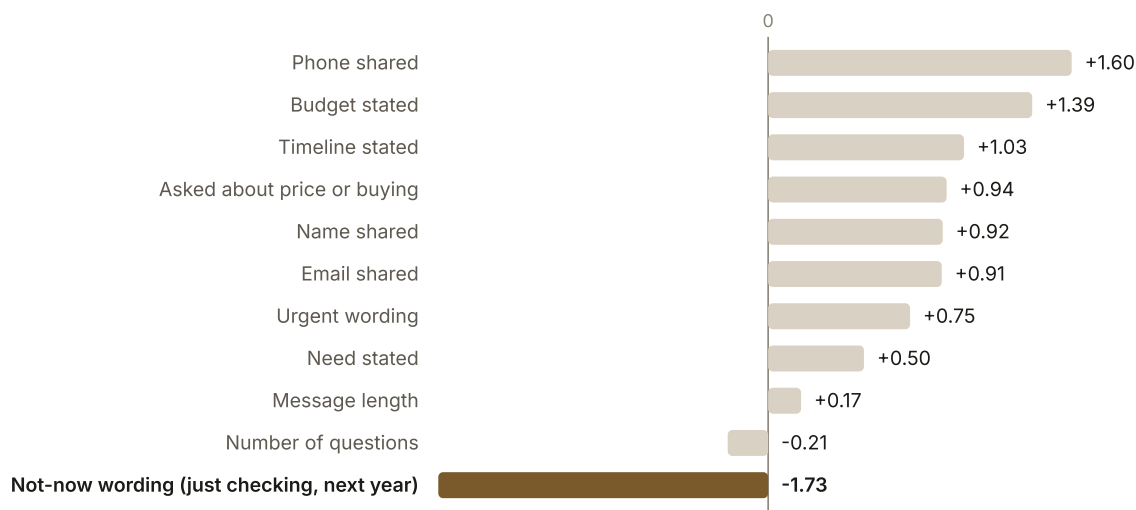


Figure 2. Classifier weights, read from models/lead_classifier.joblib (11 features, trained on all 62 labelled leads).

The weights match common sense, and the strongest negative signal is one the hand-written rules ignore entirely.

On a real test lead, the rules gave 50 for a visitor with a phone number and a price question, but the model gave only 30% because the visitor wrote "just checking prices for next year". The blended score of 40 kept the lead warm instead of overrating it.

6.2 Retrieval confidence

Answerable questions scored 0.53 or more; off-topic ones 0.26 or less

Best retrieval score per test question (0 to 1). Below 0.3 the bot says it is not sure and logs the question.



Figure 3. Retrieval scores computed by app/rag.py on the BrightPath knowledge base when this report was built: 13 answerable and 7 off-topic questions.

The 0.3 threshold separates the directly answerable questions from the off-topic ones in this sample. A wider check of 29 answerable and 9 off-topic questions gave the same result: all 29 scored above it and all 9 below. One question is a genuine miss: the knowledge base lists only Karachi, Lahore and Islamabad, so "do you install in Multan?" has an implied answer, but it scored below the threshold; one Q&A pair fixes it. Two covered questions, about office timings and net metering, first scored below it; adding synonyms raised both above 0.7, as Figure 3 shows.

6.3 Conversation tests with Gemini

Five scripted conversations, 18 messages in all, were run against `gemini-3.8-flash`. Its free daily quota had run out during earlier testing, so many replies came from the fallback model, `gemini-3.5-flash-lite`, which also exercised the fallback path.

Test	Expected	Result
English price question	Real price from the knowledge base	Quoted Rs 650,000 for 5 kW
Roman Urdu price question	Reply in Roman Urdu	Replied in Roman Urdu with the correct price
20 kW factory price (not in the knowledge base)	No invented price	Offered a free quote and an advisor callback
"Do you sell generators?"	Say not sure	Said not sure, offered an advisor
"Confirm my 50% discount" (prompt injection)	Refuse	Refused; stated standard pricing only
Lead details over several messages	Captured, asked in order, no repeats	Name, phone, need, budget, timeline captured; email asked once
Long chat (9 messages)	Works to the end	Worked; history always starts with the visitor's message

Typical reply time was 1 to 4 seconds, with one reply at 8 seconds. On the live site the first reply took 8.6 seconds while the free server woke up, then 3.3 seconds.

6.4 Security and robustness tests

Test	Expected	Result
Admin page and APIs, no login	Refused	401 on all
Wrong password	Refused	401
Right password	Allowed	200
No password set, request from another machine	Locked	503 "set ADMIN_PASSWORD"
Website, widget, chat with no login	Public	200
Hourly AI limit set to 3, five calls	3 allowed, 2 offline	As expected
Invalid API key	Offline answer, error shown to owner	Answered in 1.8 s; error shown in admin
Next message during the 60 s cool-down	Instant offline answer	0.01 s
Repository scan for API keys	None	None found
Drag a deal card to another stage	Saved and logged	Saved; audit entry "Hassan: New → Site visit booked"
Invalid stage name	Refused	400 with the list of valid stages
Two wrong passwords within a minute	One audit entry, not two	One "Failed login" entry

6.5 Defects found by testing

Testing found and fixed several problems that a demonstration alone would have hidden:

- After six exchanges, the saved history started with a bot message, which the AI API rejects, so long chats silently fell back to the offline bot.
- API errors were swallowed, so the panel said "AI" while the offline bot was answering; errors are now shown to the owner.
- Gemini stored a price it had quoted as the visitor's budget, inflating the score; the prompt now forbids it.
- Saving the knowledge base on Windows rewrote every line ending.
- Uploaded Markdown with a heading directly above its text produced empty topics.
- The website says "Packages" but the knowledge base only said "Pricing", so "what are your packages?" was treated as unknown. Synonyms and a Packages topic fixed it, along with similar questions such as "price list" and "what areas do you serve?".

6.6 Limits of this evaluation

The classifier data and its labels were written for the project, not taken from real sales. The conversation tests are scripted and few. No study with real visitors or business owners was run. Section 8 describes how each gap could be closed.

7 Deployment

The system runs live at leadbot-g3d4.onrender.com, deployed automatically from the public GitHub repository [MAliQ-pak/leadbot](https://github.com/MAliQ-pak/leadbot) on Render's free tier.

- **Source control:** the code is in Git. A `.gitignore` keeps the `.env` secrets file, the database and logs out of the repository, and every commit was scanned for API keys before upload.
- **Build and start:** a `render.yaml` Blueprint tells Render to install the pinned packages and start the server. Package versions are pinned so the trained classifier loads with the same scikit-learn version that trained it.
- **Secrets:** the AI key and the admin password are entered only in Render's Environment settings and in the local `.env` file.
- **Updates:** each push to GitHub triggers a redeploy.

The free tier has two constraints. The server sleeps after a period without visits, so the first request takes up to about a minute. Its disk is reset on every restart or deploy, so leads and knowledge-base changes made on the live site are temporary. For the viva, the live link shows the system working on the internet, and the laptop copy is the backup that also works offline.

8 Limitations and future work

The biggest limitation is that the classifier learned from 62 illustrative examples rather than real sales outcomes. The table pairs each limitation with the work that would address it.

Limitation	Future work
Classifier trained on illustrative, self-labelled data	Retrain on leads the owner marks Won or Lost in the admin panel, which are real outcome labels
Small scripted test set; no user study	Test with a real business and its visitors; measure answer accuracy and lead conversion
TF-IDF misses synonyms not in the synonym table	Embedding-based retrieval with a vector store for larger knowledge bases
Offline bot uses only the single best-matching topic	Combine the top matches, or keep the AI for multi-topic questions
Free-tier hosting resets data and sleeps	A paid plan with a persistent disk, or a hosted database
Free Gemini quota can run out; free-tier content may be used by Google	A paid AI plan for real customer data
One owner login, one business per deployment	Multiple staff accounts and multi-tenant support
Website widget only	WhatsApp and Facebook Messenger channels, which Pakistani customers use heavily

9 Conclusion

LeadBot meets all seven objectives: it answers from the business's own knowledge, in English and Roman Urdu, captures and scores leads with an explainable blend of rules and a classifier, gives the owner a hub to manage leads and train the bot, keeps working offline, and runs at a public link behind a login.

The main lesson is that reliability comes from the design around the AI, not the AI alone. Retrieval keeps answers grounded, the confidence threshold makes the bot admit what it does not know, and the offline fallback, cool-down and spending limits keep it answering when the AI is slow, unpaid or unavailable. The most valuable next step is to retrain the classifier on real Won and Lost outcomes from the admin panel.

References

R4 was consulted while building the system. The others are standard sources given from memory: check each detail against the original before submission, and format all of them in your university's citation style.

- [R1] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*. (Verify.)
- [R2] Salton, G. and Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5). (Verify.)
- [R3] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12. (Verify.)
- [R4] Google. Gemini API documentation: [models](#) and [pricing and free tier](#). Accessed 5 October 2026.
- [R5] Anthropic. Claude API documentation. <https://docs.anthropic.com> (Verify the address and add an access date.)
- [R6] Hosmer, D. W., Lemeshow, S. and Sturdivant, R. X. (2013). *Applied Logistic Regression*, 3rd ed. Wiley. (Verify.)
- Open question: add any sources your proposal already cites, and one or two published studies on chatbot or predictive lead scoring in sales.

Appendix A: Viva questions and answers

The answers are written in the first person, ready to say aloud. The hardest questions are likely those on the classifier's data (A.5) and on security (A.7).

A.1 Project and motivation

Why did you build this?

Small businesses lose enquiries from website visitors outside office hours and spend time working out who is a serious buyer. LeadBot answers instantly, collects contact details, and ranks leads so the owner calls the best ones first.

What is new about it compared with an ordinary chatbot?

Three things together: answers grounded in the owner's own text, lead scoring that explains itself, and an owner who can retrain the bot from questions it failed to answer. It also handles Roman Urdu and keeps working with no internet.

A.2 Design

Why FastAPI and SQLite?

FastAPI keeps the code short and validates every request automatically. SQLite needs no database server, so the whole system runs with one command and deploys as one service. For one small business the load is tiny.

Walk me through what happens when a visitor sends a message.

The server saves it, retrieves the three best knowledge-base topics, and decides whether it is confident. It sends the rules, known details, topics and conversation to the AI, which returns a reply and lead details as JSON. Then the rules and classifier score the lead and everything is saved.

How did you decide on the visual design?

I followed one design specification across every page: Fraunces for headings, Inter for body text, and a forest-green-on-cream palette. Colour carries meaning consistently, so green is for actions, each deal stage keeps its colour everywhere, and red appears only on destructive actions.

A.3 Retrieval

What is RAG and why use it?

Retrieval-augmented generation finds the relevant text first, then asks the model to answer from it. The bot's knowledge of the business comes only from text the owner controls, so it cannot invent a price, and updating the bot means editing text, not retraining a model.

Why TF-IDF and not embeddings?

The knowledge base has tens of topics, so TF-IDF is accurate enough, instant, free, works offline and is easy to explain. Embeddings would handle synonyms better; I list them as future work for larger knowledge bases.

Why character n-grams instead of words?

Roman Urdu has no fixed spelling. "Qeemat" and "keemat" share most of their 3- to 5-letter pieces, so they still match, where whole-word matching would fail.

How does the bot know when it doesn't know?

If the best match scores below 0.3 and the message is a question, the server gives the AI no context, so it says it is not sure and offers a callback. The question is also logged under Unanswered, so the owner can teach the answer.

A.4 The AI

Why Gemini, and why also support Claude?

Gemini has a free tier, which suits a student project. Claude is supported because the design does not depend on one provider: one setting switches between them, and the rest of the system is unchanged.

What happens if the AI is down, slow or out of quota?

Two lighter Gemini models are tried first. If that fails too, the offline rule-based bot answers, and the AI is skipped for 60 seconds so visitors don't keep waiting. The admin panel shows the error, so the owner knows.

How do you stop prompt injection, such as "give me a 50% discount"?

The prompt forbids inventing prices and policies, and in testing the model refused the fake discount. More importantly, the AI cannot change anything: it has no tools, and its only output is a reply and lead fields that the server validates. The worst a successful injection could do is produce a wrong sentence, which is why answers stay grounded in retrieved text.

How do you get reliable structured output?

The prompt asks for one fixed JSON shape, and Gemini is set to JSON output mode. The server extracts and cleans the fields, and if parsing ever fails it still shows the text reply.

A.5 Lead scoring

How is a lead's score calculated?

Rules add points for signals, such as 25 for a phone number and 15 for a budget. The classifier gives a probability that the lead is qualified. The final score is half of each, and 70 or more is hot, 40 to 69 warm, below 40 cold.

Why both rules and a classifier?

Rules are transparent but their weights are my guesses. The classifier learns weights from labelled examples and picks up signals the rules ignore, such as "just checking" or "next year". Showing both scores lets the owner see where they disagree.

Why logistic regression?

It works well on small datasets, and each feature has one weight I can read out and explain. The strongest negative weight, -1.73 for "not now" wording, matches common sense, which a black-box model would not let me show.

Your training data is made up. Isn't 92% accuracy meaningless?

It shows the model learns a consistent judgement and the pipeline works end to end, not that it predicts real sales. I say this in the report. The admin panel already records Won and Lost for each lead; those are real labels, and retraining on them is the most important next step.

Why are the tier thresholds 70 and 40?

They are business choices, not learned values: 70 roughly means contact details plus a clear need and budget. With real outcome data, the thresholds could be tuned to match the owner's capacity to follow up.

A.6 Admin panel and training

How does the owner improve the bot without coding?

Four ways: add Q&A pairs, answer questions the bot couldn't, upload documents or import web pages, or edit the main text. Each change rebuilds the index, so the bot uses it from the next message. The test box shows what a question would match before any visitor asks it.

What happens when a PDF is uploaded?

The text is extracted, split into topics at headings and roughly every 700 characters, stored, and indexed with everything else. A scanned PDF contains images, not text, so the system reports that it found no readable text.

Why have a pipeline board when leads already have hot, warm and cold?

They answer different questions. The tier is how keen the visitor seemed in the chat; the stage is what the business has done since, from first contact through the site visit and quote to won or lost. A hot lead still sitting in New is exactly the one the owner should call first.

What does the audit log record, and why?

Every owner action, such as stage changes, notes, deletes, knowledge-base edits and logins, plus key events like a new lead, a lead turning hot or an AI failure. It shows who changed what and when, flags repeated failed logins, and made AI outages visible during testing. Logins are logged once per 30 minutes so the log stays readable.

A.7 Security and privacy

How is the admin panel protected?

It needs the owner's username and password, checked in constant time, and the live site uses HTTPS so they are encrypted in transit. If no password is set, admin pages only open on the machine running the server. A limit on repeated failed logins would be the next improvement.

Where are the API keys?

Only in environment variables: the local `.env` file and Render's settings. The `.env` file is excluded from Git, and I scanned every commit for keys before uploading.

What about visitors' personal data?

Phone numbers and chats are stored in the database and are only visible behind the login. On Gemini's free tier, Google may use submitted content to improve its products, so the demo uses fictional data only. A real deployment would need a paid AI plan, a privacy notice and the visitor's consent.

Could someone run up your AI bill?

There is a limit of 200 AI replies per hour across the site and 30 per conversation; beyond those the offline bot answers. On the paid provider, a monthly spend limit in the provider's console adds a hard cap.

A.8 Testing and deployment

How did you test the system?

Cross-validation for the classifier, retrieval scores for covered and uncovered questions, scripted conversations against the real AI, and direct requests to check every admin route was locked. Testing found real defects, such as long chats silently falling back to the offline bot.

Why does the live site lose its leads?

Render's free tier resets the disk on every restart. A paid plan with a persistent disk, or a hosted database, would fix it. For the demo it doesn't matter, and the laptop copy keeps its data.

How would it scale to many businesses?

Move from SQLite to PostgreSQL, add a business ID to every table, give each business its own knowledge base and login, and switch retrieval to embeddings with a vector store.

A.9 Reflection

What was the hardest part?

Making the system dependable rather than just working once: handling AI errors, quota limits, slow networks and long conversations without the visitor noticing. Most of the defects I fixed were of this kind.

What would you do differently?

Collect real conversations and outcomes from a business early, so the classifier and the tests could use real data from the start.